Homework Assignment 1: Search Algorithms

Student Name: Farida Olumo
Student ID No: 21178511
Class Instructor: Prof. Pascal Poupart

1- Informed Search

Question 1 - Answers

1a. I expect the Manhattan distance heuristic to perform the best or better. The reason being that even though both heuristics are admissible which means they do not overestimate the cost to the goal o in other words a tighter estimate of the cost to the goal all the while still being admissible, leading A* to use fewer nodes on average. It is also known that Manhattan distance **h2(s)** will always be greater than or equal to the Misplaced Tile **h1(s)**. The concluding reason of why Manhattan tiles would perform better is because of the search efficiency, and since Manhattan is the more informed heuristic, it will prune more of the state space resulting in fewer node expansions and quicker solution.

1b. Yes, both Misplaced tile and Manhattan distance heuristics are both admissible and Consistent for the 8-puzzle.

The consistency(monotonicity) proof:

-h(n)c(n,n)+h(n)

From the Misplaced Tiles proof below, for any one move that is made only one tile changes position, ideally if the move moves just one tile into the blank. As seen below that move can reduce h1 by 1 at most.

cost of the move (c) = 1

- replacing c(n,n) with c, will yield, (changes by at almost 1)

h1(n)h1(n)1

- this can be rearranged to,

h1(n)1+h1(n)

From the proof below, a single tile changes it's Manhattan distance per step, ideally closer or farther from its goal. As seen below that can move a square either vertically or horizontally.

cost of the move (c) = 1

- replacing c(n,n) with c, will yield, (decreasing the heuristic at almost 1)

h2(n) h2(n)1

- this can be rearranged to,

h2(n)1+h2(n)

2- Constraint Satisfaction

Question 2 - Answer

2a. The Time and Space Complexity of IDA*.

Time complexity repeatedly conducts depth-first searches while progressively increasing the search limit, its worst-case running time increases exponentially.

O(b^d)

where d is the solution depth node and b is the branching factor, just as A*.

Space complexity is the main advantage of IDA*, which only stores the current path which is close in similarity to depth-first search.

O(d),

where d is the solution depth.

2b. Yes, I do believe IDA* is complete. This is complete under certain assumptions such as;

On the condition that, the branching factor has to be finite and the step cost is greater than a small positive value, it will reach finite-cost solution as long as the threshold increases, to find the goal state.

Yes, I do believe IDA* is optimal. This is optimal under certain assumptions such as;

- If h is admissible, the first solution discovered when it gets to optimal solution cost must be optimal. In other words, The first solution it discovers is assured to be optimal, because it uses an admissible heuristics and extends nodes in increasing order.

2c Table showing the Average of time and nodes along with their respective standard deviations.

Time

	B	B+FC	B+FC+H
Easy	0.008835 ± 0.004968	0.042973 ±0.0025423	0.51213 ± 0.331594
Medium	0.005512± 0.003365	0.031643 ±0.016193	0.05534 ±0.027697
Hard	0.152002± 0.076748	0.748539 ±0.436442	1.20035 ±1.179072
Evil	0.101352 ± 0.061919	0.647848 ±0.361768	3.83071 ± 3.725367

Number of Nodes

	B	B+FC	B+FC+H
Easy	936.13 ± 527.69	917.30 ± 529.77	6971.20 ± 4613.36
Medium	585.23 ± 352.26	676.50 ± 357.37	676.97 ± 324.78
Hard	16751 ± 8504.02	16169.73 ± 9268.12	16161.20 ± 15876.31
Evil	11120.87 ± 6888.72	13461.30 ±7518.44	53023.87 ±51600.65

Figure 1: Image of table

2d. Discussion of my Algorithm performance

- The following coding function that were used was referenced from the CS C384 University of Toronto Lecture slides (Hahmann, 2011). At the beginning of computing the code, I ran into some issues with the backtracking stalling and not processing because of the number of runs I had to do. The 50 runs were reduced to 30 because the backtracking had issues loading because of how slow each puzzle will take to solve on the Visual Code terminal, hence the number of runs being 30 as this number was appropriate. Regarding the step progression of the code, the search was divided into three categories. The first one was just backtracking search which was the slowest of them all to run, then the second was backtracking search and forward checking which worked well together to identify the number in the cell and cancel the possibilities for future reference. The last and formerly the fastest was the backtracking search with forward checking and heuristics. First, the sudoku board was first set up and identifies the possible number for every cell. It uses the Minimum remaining variables to look for the cells with the fewest options better known as the "hardest" square first. If two squares are equally hard, the degree heuristics breaks that tie and goes for the cell that would cause a chain reaction or in better words, influence the neighboring cells. Once the square is chosen, it uses the Least Constraining variable to sort the number that "bothers" the neighboring cells. Once the number is chosen, it validates the number to ensure it is appropriate. Once the number is placed, forward checking analyses the neighboring cells and ensures that number is "off limits" for future allocation. In the circumstance that the number hits a dead end, it makes the cell backtrack and run the process of picking a new number again. I noticed that the reason why the nodes in all searches was different for all 30 runs was because of the random value order, but faster because I had forgotten the random variable order.

References

Hahmann, T. (2011). • Some slides are modified or unmodified slides provided by Russell and Norvig. *Introduction to Artificial Intelligence*.